

CODE ANALYST REPORT

SEPTEMBER 2026



FUNCTIONAL PROGRAMMING

MODUL 1 - Functional Programming using Python

COMPILED BY:

RICO SHANDIKA JOVIAL AGISTA

202410370110433

Functional Programming G

1. Variable

10 Point

Exercise 1 : The Shape-Shifting Variable

Based on the output of the given experiment, explain how Python behaves when variables (`student_id` and `semester`) are reassigned to a completely different data type. Why do you think this flexibility can be both helpful and dangerous?

```

1  # Nama: Rico Shandika Jovial Agista
2  # NIM: 202410370110433
3
4  name = "Rico Shandika Jovial Agista"
5  student_id = "202410370110433"
6  semester = 4
7
8  print(f"Name: {name} | Type: {type(name)}")
9  print(f"ID: {student_id} | Type: {type(student_id)}")
10 print(f"Semester: {semester} | Type: {type(semester)}")
11
12 # Redefine
13 student_id = '123456'
14 semester = False
15 print(f"ID: {student_id} | Type: {type(student_id)}")
16 print(f"Semester: {semester} | Type: {type(semester)}")

```

Before:

```

ID: 123456 | Type: <class 'str'>
Semester: False | Type: <class 'bool'>

```

After redefine variable:

```

Name: Rico Shandika Jovial Agista | Type: <class 'str'>
ID: 202410370110433 | Type: <class 'str'>
Semester: 4 | Type: <class 'int'>

```

Explanation :

Python menganut dynamic typing, sehingga variabel tidak terikat pada satu tipe data. Ketika `student_id` diubah dari string menjadi string lain, dan `semester` dari integer menjadi boolean, Python langsung mengganti tipe data tanpa error. Fleksibilitas ini membantu karena kode lebih ringkas dan cepat dikembangkan, tetapi berbahaya karena bisa menimbulkan bug jika tipe data berubah tanpa disadari (misalnya string

yang seharusnya angka malah dipakai dalam perhitungan).

2. Data Type INT & Float

15 Point

Exercise 2 : Investigating Int and Float Memory Size

Modify the values in the **Memory Size Experiment** using another combination number then run the cell again and observe the differences in the memory sizes they needed. Next, perform an experiment for the float data type.

byte	digit
28	-123456789 s/d 123456789
32	-1234567890 s/d 123456789012345678
36	-1234567890123456780 s/d ?
?*	?

*) Byte values are flexible and do not need to be sequential or in multiples of 4

Does the byte size of a float increase as the value grows larger, similar to the int data type? Furthermore, why do int values have varying memory sizes?

```

1  # Nama: Rico Shandika Jovial Agista
2  # NIM: 202410370110433
3  # Integer
4
5  import sys
6
7  data1 = 123456789
8  data2 = 1234567890
9  print('Value =', data1, '| Type =', type(data1), '| Size =', sys.getsizeof(data1), 'bytes')
10 print('Value =', data2, '| Type =', type(data2), '| Size =', sys.getsizeof(data2), 'bytes')

```

#Experiment a (int)

```

Value = 123456789 | Type = <class 'int'> | Size = 28 bytes
Value = 1234567890 | Type = <class 'int'> | Size = 32 bytes

```

```
# Nama: Rico Shandika Jovial Agista
# NIM: 202410370110433
# Float
❖
import sys

f1 = 12345.6789
f2 = 1234567890.123456
print('Value =', f1, '| Type =', type(f1), '| Size =', sys.getsizeof(f1), 'bytes')
print('Value =', f2, '| Type =', type(f2), '| Size =', sys.getsizeof(f2), 'bytes')
```

#Experiment b (float)

```
Value = 12345.6789 | Type = <class 'float'> | Size = 24 bytes
Value = 1234567890.123456 | Type = <class 'float'> | Size = 24 bytes
```

Explanation :

- Integer (int): Ukuran memori bertambah seiring besarnya nilai karena Python menyimpan integer sebagai objek variable-length (menggunakan basis 2^{30}). Semakin besar angka, semakin banyak "digit" yang dibutuhkan.
- Float: Ukuran memori tetap (biasanya 24 byte di Python 64-bit) karena float direpresentasikan sebagai double-precision floating point dengan ukuran tetap, tidak peduli seberapa besar atau kecil nilainya.

3. Tuple

10 Point

Exercise 3 : Tuple Syntax Challenge

1. Can you identify why Experiment A fails while the second one runs successfully?
2. What does this tell you about how Python handles tuple creation versus tuple modification?

Eksperimen a :

```
# Nama: Rico Shandika Jovial Agista
# NIM: 202410370110433

# Experiment a (gagal)
screen_resolution = (1920, 1080)
screen_resolution[0] = 2560 # TypeError: 'tuple' object does not support item assignment
❖
```

```
Traceback (most recent call last):
  File "c:\Users\Rico\Desktop\Pemrograman-Fungsional\modul_1\Exercise-3-Tuple_Syntax_Challenge.py", line 6, in <module>
    screen_resolution[0] = 2560 # TypeError: 'tuple' object does not support item assignment
    ~~~~~^~~~~~
TypeError: 'tuple' object does not support item assignment
```

Eksperimen b:

```
# Nama: Rico Shandika Jovial Agista
# NIM: 202410370110433

# Experiment a (gagal)
#screen_resolution = (1920, 1080)
#screen_resolution[0] = 2560 # TypeError: 'tuple' object does not support item assignment

# Experiment b (berhasil)
screen_resolution = 1920, 1080
print(screen_resolution)
print(type(screen_resolution))
```

```
(1920, 1080)
<class 'tuple'>
```

Explanation :

- Experiment A gagal karena tuple bersifat immutable – setelah dibuat, elemennya tidak bisa diubah. Percobaan `screen_resolution[0] = 2560` memicu `TypeError: 'tuple' object does not support item assignment`.
- Experiment B berhasil karena hanya membuat tuple baru (tanpa mengubah isinya). Python juga mengizinkan pembuatan tuple tanpa tanda kurung, cukup dengan memisahkan nilai menggunakan koma. Ini menunjukkan bahwa Python membedakan antara pembuatan tuple (boleh tanpa kurung) dan modifikasi tuple (tidak diizinkan sama sekali).

4. Range

10 Point

Exercise 4 : Countdown Range Analysis

Create a countdown sequence using `range()` (without using if-else statements) where the starting point is determined by the last 2 digits of your Student ID (NIM).

For ODD NIM: Count down from your NIM's last 2 digits down to 1, skipping 2 numbers at a time (decrement by -2). Save and display the result as a List.

For EVEN NIM: Count down from your NIM's last 2 digits down to 0, skipping 3 numbers at a time (decrement by -3). Save and display the result as a Tuple.

Explain the logic behind the code you have created.

```
# Nama: Rico Shandika Jovial Agista
# NIM: 202410370110433 (Ganjil)

# ODD: count down dari 33 ke 1, step -2, simpan sebagai List
countdown = list(range(33, 0, -2))
print(countdown)

[33, 31, 29, 27, 25, 23, 21, 19, 17, 15, 13, 11, 9, 7, 5, 3, 1]
```

Explanation :

range(33, 0, -2) menghasilkan urutan angka mulai dari 33, turun 2 setiap langkah, berhenti sebelum 0 (yaitu 1). Fungsi list() mengubah objek range menjadi list. Karena NIM ganjil, outputnya adalah [33, 31, 29, 27, 25, 23, 21, 19, 17, 15, 13, 11, 9, 7, 5, 3, 1].

5. Dictionary

10 Point

Exercise 5 : Working with Dictionaries

1. When you updated the "semester" key in step 3, did Python create a duplicate key or did it overwrite the old value?
2. What is the explicit error message provided by Python when you uncomment and run step 4? What does that error mean?

Screenshot of the answer (must include student ID / NIM)

```
# Nama: Rico Shandika Jovial Agista
# NIM: 202410370110433

student_info = {
    "name": "Rico Shandika Jovial Agista",
    "nim": "202410370110433",
    "semester": 4
}

student_info["major"] = "Informatics"
student_info["semester"] = 5
print("Current Dictionary:", student_info)

Current Dictionary: {'name': 'Rico Shandika Jovial Agista', 'nim': '202410370110433', 'semester': 5, 'major': 'Informatics'}
```

```
# Nama: Rico Shandika Jovial Agista
# NIM: 202410370110433

student_info = {
    "name": "Rico Shandika Jovial Agista",
    "nim": "202410370110433",
    "semester": 4
}

student_info["major"] = "Informatics"
student_info["semester"] = 5
print("Current Dictionary:", student_info)
❖❖
print(student_info["gpa"]) # KeyError: 'gpa'
```

```
Current Dictionary: {'name': 'Rico Shandika Jovial Agista', 'nim': '202410370110433', 'semester': 5, 'major': 'Informatics'}
Traceback (most recent call last):
  File "c:\Users\Rico\Desktop\Pemrograman-Fungsional\modul_1\Exercise-5-Working_with_Dictionaries.py", line 14, in <module>
    print(student_info["gpa"]) # KeyError: 'gpa'
    ~~~~~^~~~~~
KeyError: 'gpa'
```

Explanation:

- Saat `student_info["semester"] = 5`, Python menimpa nilai lama (4) menjadi 5. Tidak ada duplikat key karena dictionary menyimpan key secara unik.
- Jika baris `print(student_info["gpa"])` di-uncomment, Python akan menampilkan `KeyError: 'gpa'`. Error ini berarti key "gpa" tidak ditemukan dalam dictionary. Kita harus menambahkan key tersebut terlebih dahulu atau menggunakan metode `.get()` untuk menghindari error.

6. Casting

15 Point

Exercise 6 : Data Type Casting

Based on the code execution, why does the NIM string successfully convert while the string "hello" fails? Explain the underlying mechanism of Python's `int()` and `float()` functions regarding literals.

```
# Nama: Rico Shandika Jovial Agista
# NIM: 202410370110433

NIM = "202410370110433"
print(int(NIM))
print(float(NIM))
❖
int("hello") # ValueError: invalid literal for int() with base 10: 'hello'

202410370110433
202410370110433.0
Traceback (most recent call last):
  File "c:\Users\Rico\Desktop\Pemrograman-Fungsional\modul_1\Exercise-6-Data_Type_Casting.py", line 8, in <module>
    int("hello") # ValueError: invalid literal for int() with base 10: 'hello'
    ^^^^^^^^^^^
ValueError: invalid literal for int() with base 10: 'hello'
```

Explanation:

String NIM berhasil dikonversi karena isinya hanya angka (202410370110433), yang merupakan literal numerik valid. Sebaliknya, string "hello" gagal karena bukan literal numerik. Fungsi int() dan float() hanya bisa mengubah string yang merepresentasikan angka (misalnya "123", "3.14"). Jika string berisi huruf atau karakter non-angka, Python melempar ValueError.

7. Function

15 Point

Exercise 7 : Fixing Indentation & Function Errors

1. What explicit error message did Python show before you fixed the indentation of the total and return lines?
2. Why is strict indentation so important when defining a function's body in Python compared to other programming languages?

Before:

```

calculate_discount(price, discount):
total = price - (price * discount)
return total

# Nama: Rico Shandika Jovial Agista
# NIM: 202410370110433

#def calculate_discount(price, discount):
#    total = price - (price * discount)
#    return total

result = calculate_discount(100000, 0.1)
print("Final Price:", result)

```

File "c:\Users\Rico\Desktop\Pemrograman-Fungsional\modul_1\Exercise-7-Fixing_Indentation_&_Function_Errors.py", line 1
 calculate_discount(price, discount):
 ^
 SyntaxError: invalid syntax

After:

```

# Nama: Rico Shandika Jovial Agista
# NIM: 202410370110433

def calculate_discount(price, discount):
    total = price - (price * discount)
    return total

result = calculate_discount(100000, 0.1)
print("Final Price:", result)

```

```
Final Price: 90000.0
```

Explanation:

- Sebelum diperbaiki, Python menampilkan error `IndentationError: expected an indented block` (atau `SyntaxError` karena `def` tidak ada). Penyebabnya: baris `total` dan `return` tidak diindentasi, dan tidak ada keyword `def`. Perbaikannya: tambahkan `def` di awal dan beri indentasi 4 spasi pada isi fungsi.
- Python wajib menggunakan indentasi untuk menentukan blok kode (fungsi, `if`, `loop`). Berbeda dengan bahasa lain yang memakai `{}`, indentasi di Python bukan sekadar kosmetik, melainkan bagian dari sintaks. Indentasi yang salah membuat Python tidak tahu baris mana yang termasuk dalam fungsi.

8. Recursive Function

15 Point

#Exercise 8 : Analyzing a Recursive Function

Explain step-by-step how `total_sum(3)` executes, and describe how the base case stops this function from calling itself forever!

```
# Nama: Rico Shandika Jovial Agista
# NIM: 202410370110433

def total_sum(n):
    print(f"Current n: {n}")
    if n == 1:
        return 1
    else:
        return n + total_sum(n - 1)

result = total_sum(3)
print("Final Result:", result)
```

```
Current n: 3
Current n: 2
Current n: 1
Final Result: 6
```

Explanation:

Eksekusi `total_sum(3)`:

1. $n = 3 \rightarrow$ cetak Current n: 3 $\rightarrow$ return $3 + \text{total_sum}(2)$
2. `total_sum(2)` $\rightarrow$ cetak Current n: 2 $\rightarrow$ return $2 + \text{total_sum}(1)$
3. `total_sum(1)` $\rightarrow$ cetak Current n: 1 $\rightarrow$ base case terpenuhi $\rightarrow$ return 1
4. Hasil akhir: $3 + 2 + 1 = 6$

Base case ($n == 1$) menghentikan rekursi karena fungsi tidak lagi memanggil `total_sum(n-1)`, melainkan langsung mengembalikan nilai 1. Tanpa base case, fungsi akan terus memanggil dirinya sendiri tanpa henti dan menyebabkan `RecursionError`.

CODE ANALYSIS ASSESSMENT

Code Analysis	Points (Total 10%)
Exercise Implementation	5
All exercises are completed correctly (cumulative points from all existing exercises)	100
Exercise is completed, but the code/output is incorrect	-5 (each exercise)
Answer does not include identity (Name/NIM)	-2 (each exercise)
Completed beyond the practicum session hours	0 (each exercise)
Practicum Report	5
Uses the official template, all explanation sections are filled in with correct and clear answers.	100
The answer is incorrect or insufficiently accurate.	-5 (each exercise)
The explanation is found to be identical to another practicum's	0 (each exercise)
Report submitted after the practicum session (overtime)	-40 (from the final score)
Not submitted	0